

BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: C Batch: 3 Practical No: 4

Roll no: 35 PRN No: 12414739 Name of Student: KHYATI KENDHE

Subject Name : Deep Learning

Problem Statement : Develop an LSTM-based model for time-series forecasting using stock price, weather, or sales datasets.

Algorithm :

1. Load the Google stock train/test CSVs (auto-downloading them if not present locally).
2. Merge train and test into one chronological dataframe and parse dates.
3. Compute EDA features (20/50-day moving averages, daily returns) and plot the raw data.
4. Extract the Open price column as the target series.
5. Fit a MinMaxScaler on the training portion only, then scale the full series.
6. Build sliding-window sequences (60 past days → predict the next day).
7. Split sequences chronologically into train, validation (last 60 days pre-test), and test (official 20-day window).
8. Reshape sequences into 3D LSTM input format (samples, timesteps, features).
9. Define a stacked LSTM model (LSTM → Dropout → LSTM → Dropout → Dense → Dense).
10. Compile the model with Adam optimizer and MSE loss.
11. Train the model with early stopping on validation loss.
12. Plot training vs validation loss curves.
13. Generate predictions on train, validation, and test sets.
14. Inverse-transform predictions and true values back to original price scale.
15. Compute evaluation metrics (RMSE, MAE, MAPE, R^2) for each split.
16. Plot actual vs predicted prices across the full timeline.
17. Plot a zoomed-in actual vs predicted comparison for the test window.
18. Plot a predicted-vs-actual scatter plot with a perfect-prediction reference line.
19. Analyze residuals over time and their distribution.
20. Assemble all key plots into one summary dashboard figure.

Code:

"""

LSTM-Based Time-Series Forecasting on Stock Price Data

Dataset : Google (GOOG) daily stock price, Jan-2012 to Jan-2017

"Google_Stock_Price_Train.csv" / "Google_Stock_Price_Test.csv"

-> THE classic stock-price dataset used in almost every LSTM

tutorial (originally distributed via the SuperDataScience

Deep Learning course, mirrored widely on GitHub & Kaggle:

<https://www.kaggle.com/datasets/akram24/google-stock-price-train>)

"""

```
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
import seaborn as sns
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
```

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout, Input
from tensorflow.keras.callbacks import EarlyStopping
```

```
# -----
# 0. CONFIG
# -----
SEED = 42
np.random.seed(SEED)
tf.random.set_seed(SEED)
```

```
TRAIN_CSV = "google_stock_train.csv" # 1258 rows: 2012-01-03 -> 2016-12-30
TEST_CSV = "google_stock_test.csv" # 20 rows: 2017-01-03 -> 2017-01-31
```

```
# Source URLs (used only if the CSVs aren't already sitting next to this script)
TRAIN_URL = "https://raw.githubusercontent.com/Debasishsaha123/Stock-prediction-
/main/Google_Stock_Price_Train.csv"
TEST_URL = "https://raw.githubusercontent.com/Debasishsaha123/Stock-prediction-
/main/Google_Stock_Price_Test.csv"
```

```
def ensure_csv(path, url):
    """Download the dataset if it isn't already present in the working directory."""
    if not os.path.exists(path):
        import urllib.request
        print(f"{path}" not found locally — downloading from {url} ...")
```

```

urllib.request.urlretrieve(url, path)
print(f"Saved to {path}")

ensure_csv(TRAIN_CSV, TRAIN_URL)
ensure_csv(TEST_CSV, TEST_URL)
TARGET_COL = "Open" # standard target for this dataset
WINDOW_SIZE = 60 # look-back window (trading days)
N_TEST = 20 # matches the official test file
N_VAL = 60 # carved out of training data for early stopping
EPOCHS = 150
BATCH_SIZE = 32
OUT_DIR = "outputs_google"
os.makedirs(OUT_DIR, exist_ok=True)

sns.set_theme(style="darkgrid", palette="deep")
plt.rcParams.update({
    "figure.facecolor": "#0f1117",
    "axes.facecolor": "#161a23",
    "savefig.facecolor": "#0f1117",
    "axes.edgecolor": "#3a3f4b",
    "axes.labelcolor": "#e8e8e8",
    "text.color": "#e8e8e8",
    "xtick.color": "#c7c7c7",
    "ytick.color": "#c7c7c7",
    "grid.color": "#2a2f3a",
    "figure.dpi": 120,
})

# -----
# 1. LOAD & MERGE DATA
# -----
train_df = pd.read_csv(TRAIN_CSV, thousands=",")
test_df = pd.read_csv(TEST_CSV, thousands=",")

df = pd.concat([train_df, test_df], ignore_index=True)
df["Date"] = pd.to_datetime(df["Date"], format="%m/%d/%Y")
df = df.sort_values("Date").reset_index(drop=True)

df["MA20"] = df[TARGET_COL].rolling(20).mean()
df["MA50"] = df[TARGET_COL].rolling(50).mean()
df["DailyReturn"] = df[TARGET_COL].pct_change() * 100

print(f"Loaded {len(df)} rows | {df['Date'].min().date()} -> {df['Date'].max().date()}")
print(df[[TARGET_COL, "Volume"]].describe())

# -----
# 2. EDA VISUALIZATION - price history, volume, moving averages, returns
# -----
fig, axes = plt.subplots(3, 1, figsize=(13, 10), sharex=True,
                        gridspec_kw={"height_ratios": [3, 1, 1]})

```

```

axes[0].plot(df["Date"], df[TARGET_COL], color="#4fc3f7", lw=1.4, label="Open")
axes[0].plot(df["Date"], df["MA20"], color="#ffb74d", lw=1.1, label="MA-20")
axes[0].plot(df["Date"], df["MA50"], color="#ba68c8", lw=1.1, label="MA-50")
axes[0].fill_between(df["Date"], df["Low"], df["High"], color="#4fc3f7", alpha=0.08)
axes[0].set_title("Google (GOOG) Opening Price with Moving Averages", fontsize=13, weight="bold")
axes[0].set_ylabel("Price (USD)")
axes[0].legend(loc="upper left", frameon=False)

```

```

axes[1].bar(df["Date"], df["Volume"], color="#607d8b", width=1.2)
axes[1].set_ylabel("Volume")
axes[1].set_title("Trading Volume", fontsize=11)

```

```

colors = np.where(df["DailyReturn"] >= 0, "#66bb6a", "#ef5350")
axes[2].bar(df["Date"], df["DailyReturn"], color=colors, width=1.2)
axes[2].set_ylabel("Daily Return (%)")
axes[2].set_title("Daily Returns", fontsize=11)
axes[2].xaxis.set_major_locator(mdates.YearLocator())
axes[2].xaxis.set_major_formatter(mdates.DateFormatter("%Y"))
plt.setp(axes[2].get_xticklabels(), rotation=0, ha="center")

```

```

fig.tight_layout()
fig.savefig(f"{OUT_DIR}/01_eda_overview.png")
plt.close(fig)

```

```

# -----
# 3. PREPROCESS - scale & window
# -----

```

```

prices = df[TARGET_COL].values.astype("float32")
n = len(prices)
n_train = n - N_TEST - N_VAL # everything before the val block

```

```

# Fit scaler ONLY on the training portion to avoid look-ahead leakage
scaler = MinMaxScaler(feature_range=(0, 1))
scaler.fit(prices[:n_train])
scaled = scaler.transform(prices)

```

```

def make_sequences(arr, window):
    X, y = [], []
    for i in range(window, len(arr)):
        X.append(arr[i - window:i, 0])
        y.append(arr[i, 0])
    return np.array(X), np.array(y)

```

```

X_all, y_all = make_sequences(scaled, WINDOW_SIZE)
dates_all = df["Date"].values[WINDOW_SIZE:]

```

```

train_end = n_train - WINDOW_SIZE
val_end = (n_train + N_VAL) - WINDOW_SIZE

```

```

X_train, y_train = X_all[:train_end], y_all[:train_end]
X_val, y_val = X_all[train_end:val_end], y_all[train_end:val_end]

```

```

X_test, y_test = X_all[val_end:], y_all[val_end:]
dates_train, dates_val, dates_test = dates_all[:train_end], dates_all[train_end:val_end],
dates_all[val_end:]

X_train = X_train.reshape(-1, WINDOW_SIZE, 1)
X_val = X_val.reshape(-1, WINDOW_SIZE, 1)
X_test = X_test.reshape(-1, WINDOW_SIZE, 1)

print(f'\nSequence shapes -> train: {X_train.shape} val: {X_val.shape} test: {X_test.shape}')

# -----
# 4. BUILD MODEL
# -----
model = Sequential([
    Input(shape=(WINDOW_SIZE, 1)),
    LSTM(64, return_sequences=True),
    Dropout(0.2),
    LSTM(64, return_sequences=False),
    Dropout(0.2),
    Dense(32, activation="relu"),
    Dense(1),
])
model.compile(optimizer="adam", loss="mse", metrics=["mae"])
model.summary()

early_stop = EarlyStopping(monitor="val_loss", patience=15, restore_best_weights=True)

history = model.fit(
    X_train, y_train,
    validation_data=(X_val, y_val),
    epochs=EPOCHS,
    batch_size=BATCH_SIZE,
    callbacks=[early_stop],
    verbose=2,
)

# -----
# 5. LOSS CURVE VISUALIZATION
# -----
fig, ax = plt.subplots(figsize=(9, 5))
ax.plot(history.history["loss"], label="Train Loss (MSE)", color="#4fc3f7", lw=2)
ax.plot(history.history["val_loss"], label="Validation Loss (MSE)", color="#ef5350", lw=2)
ax.set_title("Training vs Validation Loss", fontsize=13, weight="bold")
ax.set_xlabel("Epoch"); ax.set_ylabel("MSE (scaled)")
ax.legend(frameon=False)
fig.tight_layout()
fig.savefig(f'{OUT_DIR}/02_loss_curves.png')
plt.close(fig)

# -----
# 6. PREDICTIONS + INVERSE SCALE

```

```

# -----
def predict_inverse(X):
    pred_scaled = model.predict(X, verbose=0)
    return scaler.inverse_transform(pred_scaled)

pred_train = predict_inverse(X_train)
pred_val = predict_inverse(X_val)
pred_test = predict_inverse(X_test)

true_train = scaler.inverse_transform(y_train.reshape(-1, 1))
true_val = scaler.inverse_transform(y_val.reshape(-1, 1))
true_test = scaler.inverse_transform(y_test.reshape(-1, 1))

# -----
# 7. METRICS
# -----
def metrics(y_true, y_pred, label):
    rmse = np.sqrt(mean_squared_error(y_true, y_pred))
    mae = mean_absolute_error(y_true, y_pred)
    mape = np.mean(np.abs((y_true - y_pred) / y_true)) * 100
    r2 = r2_score(y_true, y_pred)
    print(f'{label}>6} | RMSE: {rmse:6.3f} MAE: {mae:6.3f} MAPE: {mape:5.2f}% R2: {r2:5.3f}')
    return rmse, mae, mape, r2

print("\n--- Performance ---")
metrics(true_train, pred_train, "Train")
metrics(true_val, pred_val, "Val")
test_rmse, test_mae, test_mape, test_r2 = metrics(true_test, pred_test, "Test")

# -----
# 8. FULL-TIMELINE ACTUAL VS PREDICTED (train/val/test shaded)
# -----
fig, ax = plt.subplots(figsize=(14, 6))
ax.plot(df["Date"], df[TARGET_COL], color="#78909c", lw=1, label="Actual (full series)", alpha=0.6)

ax.plot(dates_train, pred_train, color="#4fc3f7", lw=1.4, label="Predicted (train)")
ax.plot(dates_val, pred_val, color="#ffb74d", lw=1.4, label="Predicted (val)")
ax.plot(dates_test, pred_test, color="#ef5350", lw=2.2, marker="o", ms=4, label="Predicted (test)")

ax.axvspan(dates_val[0], dates_val[-1], color="#ffb74d", alpha=0.06)
ax.axvspan(dates_test[0], dates_test[-1], color="#ef5350", alpha=0.10)

ax.set_title("LSTM Forecast vs Actual Opening Price (Full Timeline)", fontsize=13, weight="bold")
ax.set_ylabel("Price (USD)")
ax.legend(loc="upper left", frameon=False, ncol=2)
fig.autofmt_xdate()
fig.tight_layout()
fig.savefig(f'{OUT_DIR}/03_full_timeline_forecast.png')
plt.close(fig)

# -----

```

```
# 9. ZOOMED TEST-SET ACTUAL VS PREDICTED (the official Jan-2017 test window)
```

```
# -----
```

```
fig, ax = plt.subplots(figsize=(11, 5.5))
ax.plot(dates_test, true_test, color="#e8e8e8", lw=2.2, marker="o", ms=6, label="Actual")
ax.plot(dates_test, pred_test, color="#ef5350", lw=2.2, marker="o", ms=6, label="Predicted")
ax.fill_between(dates_test, true_test.flatten(), pred_test.flatten(),
               color="#ef5350", alpha=0.15)
ax.set_title(f'Jan-2017 Test Set: Actual vs Predicted (RMSE={test_rmse:.2f},
MAPE={test_mape:.2f}%)',
            fontsize=13, weight="bold")
ax.set_ylabel("Price (USD)")
ax.legend(frameon=False)
fig.autofmt_xdate()
fig.tight_layout()
fig.savefig(f'{OUT_DIR}/04_test_zoom_forecast.png')
plt.close(fig)
```

```
# -----
```

```
# 10. SCATTER: PREDICTED VS ACTUAL (with y=x reference)
```

```
# -----
```

```
fig, ax = plt.subplots(figsize=(6.5, 6.5))
ax.scatter(true_test, pred_test, color="#4fc3f7", alpha=0.85, edgecolor="#0f1117", s=70)
lims = [min(true_test.min(), pred_test.min()), max(true_test.max(), pred_test.max())]
ax.plot(lims, lims, color="#ef5350", lw=1.5, ls="--", label="Perfect Prediction")
ax.set_xlim(lims); ax.set_ylim(lims)
ax.set_xlabel("Actual Price (USD)"); ax.set_ylabel("Predicted Price (USD)")
ax.set_title(f'Predicted vs Actual (Test Set)  $R^2 = \{test\_r2:.3f\}$ ', fontsize=12, weight="bold")
ax.legend(frameon=False)
fig.tight_layout()
fig.savefig(f'{OUT_DIR}/05_scatter_pred_vs_actual.png')
plt.close(fig)
```

```
# -----
```

```
# 11. RESIDUAL ANALYSIS
```

```
# -----
```

```
residuals = (true_test - pred_test).flatten()

fig, axes = plt.subplots(1, 2, figsize=(13, 4.8))
axes[0].plot(dates_test, residuals, color="#ba68c8", lw=1.6, marker="o", ms=5)
axes[0].axhline(0, color="#e8e8e8", lw=1, ls="--")
axes[0].set_title("Residuals Over Time (Test Set)", fontsize=12, weight="bold")
axes[0].set_ylabel("Actual - Predicted (USD)")
plt.setp(axes[0].get_xticklabels(), rotation=30, ha="right")

sns.histplot(residuals, kde=True, ax=axes[1], color="#4fc3f7", edgecolor="#0f1117", bins=10)
axes[1].axvline(0, color="#ef5350", lw=1.5, ls="--")
axes[1].set_title("Residual Distribution", fontsize=12, weight="bold")
axes[1].set_xlabel("Residual (USD)")

fig.tight_layout()
fig.savefig(f'{OUT_DIR}/06_residual_analysis.png')
```

```
plt.close(fig)
```

OUTPUT:







